LatchMoE: Graph-Compatible Expert Offloading through Address-Stable Residency

Anonymous Author(s)

Abstract

Sparse mixture-of-experts models bound per-token arithmetic but retain a large parameter footprint. Expert offloading addresses this footprint, yet a bounded expert cache changes weight placement as routing changes, conflicting with the stable storage bindings expected by graph replay. Our key insight is that replay requires stable graph-visible storage, not static logical expert residency. We present LatchMoE, which stages routed experts across an eager seam into fixed device slots, executes oversized working sets in capacity-bounded waves, and uses versioned leases to order transfer readiness, mapping publication, computation, and reuse. A vLLM-Ascend implementation evaluates the three target models under 11 semantic, graph, and lifecycle gates; Qwen3-Next's native full-resident end-to-end subcheck is capacity-limited on this device. On Qwen3-30B-A3B, bounded waves change paired TTFT by -35.13% in an internal comparison against graph-compatible full-layer staging; its 95% CI is [-35.67%, -34.60%]. Enabling overlap in a one-factor campaign changes TTFT by -27.87%; its 95% CI is [-30.18%, -25.24%]. Performance results are scoped to one Ascend 910B2, BF16, tensor parallelism one, and serving concurrency one. Against full residency, the same workload trades a median sampled HBM peak from 97% to 80% for +261.28% TTFT and +309.51% TPOT; the reported TTFT reductions therefore characterize capacity-bounded staging rather than a universal end-to-end speedup.

1 Introduction

Mixture-of-Experts (MoE) architectures have emerged as a prominent approach for scaling Large Language Models (LLMs) to hundreds of billions of parameters [2, 5, 8, 16]. By dynamically routing each token to only a sparse subset of experts, MoE models substantially increase model capacity while keeping per-token computation bounded [3, 15]. However, sparse activation reduces computation rather than model storage: although only a few experts are executed for a given token, the weights of all experts must remain accessible because any expert may be selected at runtime. As MoE models scale, the complete expert pool can exceed accelerator High-Bandwidth Memory (HBM) capacity. To serve such models on memory-constrained accelerators, **expert offloading** has become an important strategy: the runtime keeps a portion of expert weights in host memory and accesses or stages them on demand during inference [7, 10, 24].

To make expert offloading practical, prior systems reduce its data-movement overhead through expert caching, routing-aware prediction and prefetching, and transfer–computation overlap [1, 4, 7, 9, 24]. These techniques reduce the frequency of host-to-device transfers or hide part of their latency. However, they do not remove the execution dynamism of cache-based expert offloading: expert admission and eviction in a bounded HBM cache can change device storage assignments across inference steps.

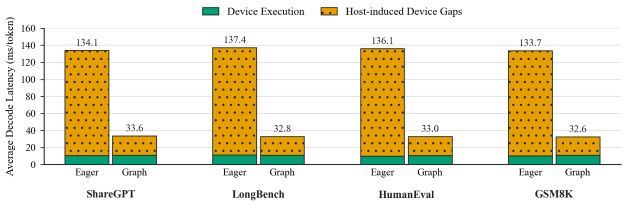

Figure 1. Average decode latency breakdown under eager execution and graph replay. Graph replay substantially reduces host-induced device gaps while leaving device execution time nearly unchanged.

Such runtime changes can be naturally accommodated under eager execution, where the host resolves the current device storage assignments and dispatches the corresponding kernels at each decoding step. This flexibility, however, comes at a substantial performance cost. Autoregressive decoding repeatedly executes a largely recurring sequence of fine-grained accelerator kernels, and launching these kernels individually incurs repeated host-side scheduling, dispatch, and synchronization overhead. **Graph replay** mitigates this overhead by capturing the recurring execution sequence once and replaying it as a unit, thereby reducing host-induced gaps between kernel executions [17, 25].

As shown in Figure 1, graph replay reduces average decode latency by approximately 75% across four workloads, from 133.7–137.4 ms/token under eager execution to 32.6–33.6 ms/token, while device execution time remains nearly unchanged. This result indicates that most of the improvement comes from eliminating host-induced device idle time rather than accelerating the underlying kernels.

However, graph replay requires its captured execution state to remain replay-stable. A dynamically managed expert cache can violate this requirement when expert admission or eviction changes the device storage assignment exposed to captured computation. A cache miss may therefore require updating or recapturing the graph to reflect new storage bindings, or falling back to eager execution.

All three options add host work and reduce the benefit of graph replay. The resulting systems problem is to **enable dynamic expert residency and device storage assignment within a bounded HBM cache while preserving the graph-visible storage bindings required by graph replay**.

UVA allows accelerator kernels to access pinned host-resident weights through device-visible addresses [13, 21]. When the backing allocations remain alive, the same addresses can be reused across graph replay [14]. UVA therefore addresses address stability for host-resident weights, but does not by itself provide routing-driven replacement within a bounded HBM expert cache. Thus, preserving graph replay under cache-based expert offloading remains challenging.

Our key insight is that **graph replay requires stable graph-visible storage bindings, not static expert residency**. A fixed set of HBM slots can provide those bindings while the runtime changes which logical experts occupy the slots. This separation allows device storage assignments to evolve without changing the bindings observed by replayed computation.

Realizing this abstraction raises three systems challenges. First, routing-dependent expert staging must be isolated from captured execution. Active experts are known only after routing, yet expert admission and eviction must not alter the graph-visible storage bindings or captured execution structure. Second, bounded slot storage must be updated without placing expert transfer entirely on the critical path. Changing routing decisions may require repeated expert movement, and the routed working set may exceed the available slots, making transfer–computation overlap essential for efficient execution. Third, slot reuse must remain correct under asynchronous transfer and computation. A slot cannot be reassigned while its previous contents are still in use, nor can a new assignment become visible before the staged weights are ready.

We present LatchMoE, a graph-compatible MoE offloading system that maps changing logical experts onto a fixed set of device slots. LatchMoE performs expert staging outside the replayed computation, allowing replayed kernels to access the same slots even as the resident experts change. For efficient reuse of bounded slot storage, LatchMoE organizes expert execution into waves and overlaps the staging of upcoming experts with ongoing computation. Finally, LatchMoE coordinates asynchronous transfer, computation, and slot reassignment through a version-controlled handoff protocol, ensuring that each slot is reused only when its previous use has completed and its new contents are ready. Together, these mechanisms enable dynamic expert residency and device storage assignment within a bounded HBM cache while preserving graph replay.

In summary, this paper makes the following contributions:

- We identify a cross-layer conflict between cache-based expert offloading and graph replay in memory-constrained MoE inference: routing-driven expert admission, eviction, and replacement cause device storage assignments to evolve, whereas graph replay reuses the graph-visible storage binding referenced by captured computation. From this conflict, we derive the key insight that replayability requires stable graph-visible storage rather than static expert residency.
- We design LatchMoE, a graph-compatible MoE offloading runtime that virtualizes dynamic logical experts over replay-stable device slots. LatchMoE isolates routing-dependent expert staging from captured computation, pipelines expert transfer with computation through wave-based execution, and coordinates safe slot reuse through a version-controlled handoff protocol.
- We implement LatchMoE on top of vLLM-Ascend and evaluate it under constrained HBM. On Qwen3-30B-A3B, LatchMoE preserves exact model outputs and graph replay under cache-based expert offloading. Separately, compared with graph-compatible full-layer staging, its multi-wave prefill execution reduces repeat-median TTFT p50 by 35.21%.

2 Background and Problem Formulation

2.1 MoE Inference and Cache-Based Expert Offloading

A Mixture-of-Experts (MoE) layer replaces the dense feed-forward network with a set of experts and a router. For each input token, the router selects a small subset of experts and combines their outputs according to the routing weights [2, 5, 8, 16]. We refer to an expert's model-defined identity as its *logical expert ID*. For an invocation of a given MoE layer at step t, let $\mathcal{A}_t$ denote the set of logical experts selected across all participating tokens. Because routing is input-dependent, $\mathcal{A}_t$ varies across inference steps. This active-set variability is especially pronounced during prefill, where many prompt tokens are processed together and may collectively activate substantially more experts than a typical decoding step [4].

When the complete expert pool does not fit in accelerator HBM, expert offloading places part of the expert weights in host memory. We focus on *cache-based expert offloading*, which maintains a bounded expert cache in HBM and stages non-resident experts from host memory on demand [1, 4, 7, 24]. Let $\mathcal{H}_t$ denote the experts of this layer currently resident in HBM, with $|\mathcal{H}_t| \leq C$, where C denotes the maximum number of experts that the configured cache can hold. A requested expert $e \in \mathcal{A}_t \cap \mathcal{H}_t$ is a cache hit, whereas $e \in \mathcal{A}_t \setminus \mathcal{H}_t$ incurs an expert cache miss and must be staged from host memory. If the cache is full, admitting a missing expert requires evicting or replacing an existing resident expert.

Cache-based offloading therefore makes expert residency and placement runtime-managed state. For each resident expert $e \in \mathcal{H}_t$, let $\pi_t(e)$ denote the device storage location assigned to its weight tensors at step t. As routing decisions trigger admission, eviction, and replacement, both the resident set $\mathcal{H}_t$ and the placement mapping π_t may evolve over time. Thus, a logical expert has a fixed model identity, while its HBM residency and physical placement are dynamic.

2.2 Graph Replay and Replay-Stable Execution State

Graph replay reduces repeated host-side scheduling and kernel-launch overhead by capturing a recurring accelerator execution sequence and replaying it as a unit [17, 25]. Ordinary graph replay reuses the captured operation structure and graph-visible storage bindings. Updating or recapturing this state can accommodate binding changes, but requires additional host intervention [14]. We refer to the state required for ordinary replay as the *replay-stable execution state*, and denote its storage bindings by $\mathcal{B}$.

Replay stability does not require runtime data to remain unchanged. Values stored in preallocated graph-visible buffers may be updated between replays as long as the captured computation continues to access the same storage. For example, FlashInfer stores request-dependent scheduling metadata in preallocated device workspaces and updates its contents while preserving the workspace addresses referenced by captured kernels [25]. This example shows that graph replay can accommodate dynamic values behind stable storage bindings, whereas binding changes require graph update or recapture.

2.3 Dynamic Expert Placement Meets Graph Replay

The conflict arises when cache-based expert offloading exposes the current device storage assignment of expert weights directly to captured computation. Consider a logical expert e that is resident at two inference steps t and $t' > t$. If e is evicted after step t and later reloaded, it may occupy a different HBM location:

$$\pi_t(e) \neq \pi_{t'}(e).$$

Such placement changes are natural under a bounded expert cache, whose contents evolve with routing decisions.

If captured expert computation directly binds e to its current HBM location, a change in $\pi_t(e)$ also changes the graph-visible binding $\mathcal{B}$. Dynamic expert caching may therefore require the resident set $\mathcal{H}_t$ and placement mapping π_t to evolve, while conventional graph replay expects its graph-visible bindings $\mathcal{B}$ to remain reusable. The desired execution abstraction must decouple these two requirements:

$$(\mathcal{H}_t, \pi_t) \text{ may evolve}, \qquad \mathcal{B} \text{ remains replay-stable.}$$

Several straightforward alternatives avoid one side of this conflict only by sacrificing another system objective. Keeping all experts permanently resident would stabilize their physical bindings but defeats offloading under a bounded HBM budget. Falling back to eager execution accommodates arbitrary placement changes but forfeits the host-overhead savings of graph replay. Updating or recapturing the graph after each placement change similarly reintroduces host intervention into the repeated inference path.

Therefore, our goal is to support dynamic expert residency and storage assignment within a bounded HBM cache while preserving the graph-visible storage bindings required by ordinary replay.

3 Motivation and Characterization

Graph-compatible offloading is necessary only if routing changes residency, bounded device storage cannot hold the resulting working set, and exploiting transfer–compute overlap makes slot reuse asynchronous. We test these conditions on Qwen3-30B-A3B, GLM-4.7-Flash, and Qwen3-Next-80B-A3B-Instruct. Runs use one Ascend 910B2, BF16, TP1, disabled prefix caching, and serialized requests (both concurrency controls set to one); thus, the measured pressure does not come from cross-request batching. We profile 12, 11, and 48 managed routed layers over 200, 100, and 64 ShareGPT requests, respectively. Qwen3 and GLM generate at most 128 output tokens; Qwen3-Next uses 32 to bound collection cost. We therefore use cross-model results only to establish whether a phenomenon recurs, not to rank its magnitude.

An independent audit reparses each raw event stream, verifies its SHA-256 and run manifest, and exactly reproduces all statistics below. It also checks balanced full-rate decode sampling, successful requests, and device-release acknowledgments. Figure 2 summarizes the two workload properties that determine storage demand.

3.1 Routing-Driven Expert Residency Is Dynamic

Decode activates only 8, 4, and 10 experts per layer in Qwen3, GLM, and Qwen3-Next. Yet a 32-slot cache misses on 20.55%, 14.02%, and 38.20% of expert accesses. More importantly, 68.71%, 41.03%, and 94.22% of layer invocations contain a miss, while 68.09%, 40.38%, and 93.90% update at least one mapping entry (Figure 2a). These are not artifacts of interleaved layers: consecutive invocations are compared within each layer. Their median active-set Jaccard similarities are only 0.333, 0.143, and 0.250. Hence, sparsity bounds per-token work but does not make logical ownership of physical storage stable.

Implication: replay-stable physical slots. Graph-visible allocations must remain fixed while their logical owners and a persistent mapping change between replays. Binding a graph directly to each active expert's original tensor cannot satisfy both requirements.

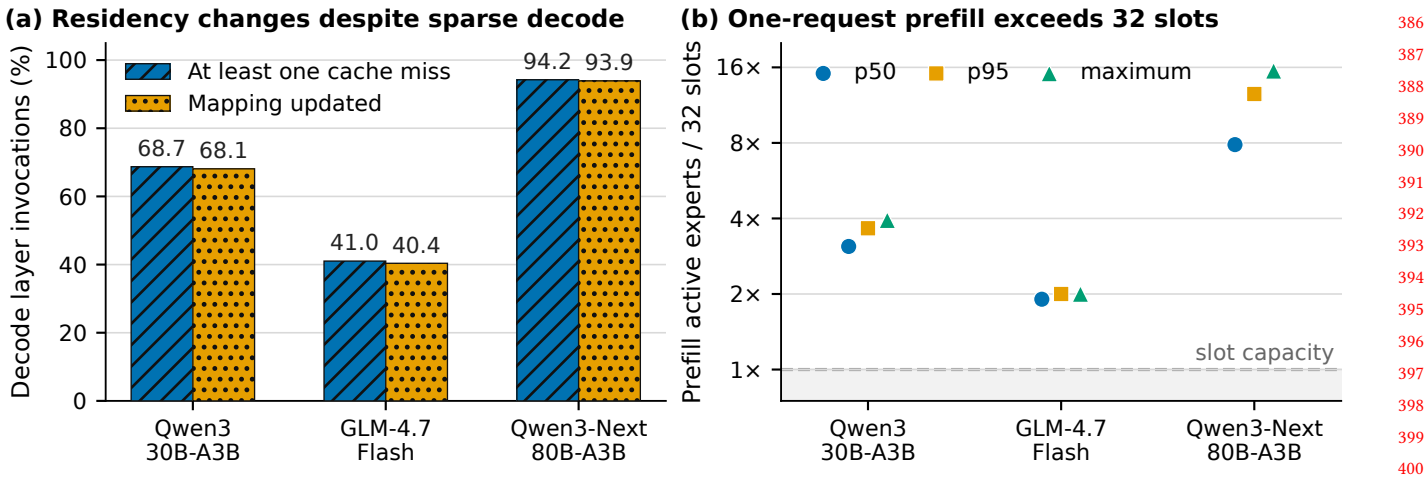

Figure 2. Routing-induced storage pressure under serialized execution. (a) Sparse decode frequently requests a nonresident expert and changes the logical-to-physical mapping. (b) The p50, p95, and maximum number of distinct routed experts in a single-request prefill, normalized by a 32-slot bank; every profiled invocation exceeds the dashed capacity line. Traces contain 200/100/64 requests and cover 12/11/48 managed layers for Qwen3, GLM, and Qwen3-Next, respectively; Qwen3-Next uses a shorter output limit. All traces use one Ascend 910B2, BF16, tensor parallelism one, disabled prefix caching, and serialized requests.

3.2 Routed Working Sets Can Exceed Bounded Slot Capacity

Prefill aggregates routing decisions across a prompt. The distinct active experts per layer have p50/p95/maximum values of 99/117/126 for Qwen3, 61/64/64 for GLM, and 252/401/496 for Qwen3-Next. Consequently, all 2,400, 1,100, and 3,072 profiled prefill invocations overflow 32 slots. Their required wave counts at p50/p95/maximum are 4/4/4, 2/2/2, and 8/13/16 (Figure 2b). This occurs with one request at a time: increasing the serving batch is not a prerequisite for overflow.

The corresponding p50 active-weight footprints are 0.87, 1.07, and 1.48 GiB, versus 32-slot budgets of 0.28, 0.56, and 0.19 GiB. Nor is 32 a special cutoff: 64 slots still overflow 99.29% of Qwen3 prefills and every Qwen3-Next prefill; 128 slots still overflow 99.48% of Qwen3-Next prefills.

Implication: wave-based slot reuse. A fixed slot space must be reused within one layer invocation. Each wave may materialize at most the slot capacity, while the union of waves must preserve every routed token–expert pair and its original combine position.

3.3 Overlapped Expert Staging Creates Slot-Reuse Hazards

Wave reuse exposes substantial transfer work. Across the scoped prefills, Qwen3, GLM, and Qwen3-Next execute 8,571, 2,200, and 26,695 waves and move 1.58, 0.60, and 4.62 TB from host memory. Later-wave prefetches are issued before compute 6,171, 1,100, and 23,623 times; the remaining issue in each invocation is its initial wave. Even with this schedule, aggregate staging wait is 1,459.9, 391.9, and 4,683.3 ms—20.2%, 24.0%, and 17.2% of the corresponding profiled MLP time. These values establish an overlap opportunity, not a causal latency benefit; Evaluation tests that claim separately.

Overlap makes transfer readiness, compute completion, and reuse of the same bank advance on different streams. Two invalid executions become possible. First, staging wave $k+2$ can overwrite a slot whose lease is still consumed by wave k. Second, publishing wave $k+1$'s mapping before its H2D completes can make incomplete weights visible; accepting a completion from an older owner creates the same failure. The correct traces contain zero premature publication events and install a consumer dependency for every scoped decode invocation, demonstrating that these conditions are observable gates rather than assumed ordering. The hazard itself is a consequence of asynchronous reuse, not an observed model error.

Implication: a versioned lease protocol. Reuse must validate the slot owner and version, publish a mapping only after the matching transfer is ready, and block reassignment until the matching compute lease releases. Section 4 turns these three pressures into one address-stable execution architecture.

4 Design

4.1 Overview

LatchMoE's central principle is that *logical expert ownership may change, while graph-visible storage remains fixed.* For each managed layer, it preallocates bounded expert slots and

a persistent logical-to-slot map. Their allocations survive capture and replay; routing changes only slot contents and map values.

Figure 3 shows the resulting captured-router, eager-stage, and captured-MLP path. Staging transfers misses and publishes the map before the MLP resumes through the same graph-visible interface.

Three mechanisms realize this model: replay-stable slots decouple residency from addresses; pipelined waves reuse bounded storage; and versioned leases order transfer, publication, computation, and reuse.

4.2 Capability-Driven Semantic Contract

Capability descriptor. Before selecting the seam, LatchMoE materializes a serializable descriptor for each managed layer. The descriptor records (i) the output contract; (ii) the shared-expert representation and activation mode; (iii) whether routing logits are externally supplied or produced by an internal gate; (iv) all top-k, grouping, scoring, bias, normalization, and scaling parameters; (v) the owner of final combination; and (vi) weight, parallelism, and overlap modes. The selection rule uses this tuple rather than a model allowlist. Routed-only and externally shared-expert layers select distinct, fixed return schemas, avoiding a data-dependent captured type.

Fail-closed selection. The seam is enabled only when every descriptor field belongs to an implemented profile. Unknown router adapters, unregistered or mismatched fused shared/routed placement, quantized weights, multi-NPU execution, and shared-expert overlap are rejected before capture with the full descriptor and blocker list; rejection never silently chooses an eager or monolithic path. Manifests therefore identify the exact accepted tuple.

4.3 Replay-Stable Dual-Lane Storage

Routed experts form a dynamic lane. For each managed layer, LatchMoE retains their W_{13} and W_2 weights in a CPU-first host store and allocates two contiguous device tensors containing a bounded number of physical expert slots. A persistent tensor maps each logical routed expert ID to a slot ID. The slot tensors and mapping allocation survive capture and replay; replacement changes only slot contents and mapping values.

The shared lane remains device-resident because it contributes to every applicable token; caching it would consume capacity without sparsity and repeat shared computation in each overflow wave. Its bytes are charged separately in the memory ledger, while capacity, victim, and wave decisions cover only routed experts.

Each routed slot records a logical owner, a monotonically increasing version, a state, and its last-use step. A *lease* is the tuple (slot ID, logical expert, version). The version distinguishes ownership intervals that reuse the same physical address. The central storage invariant is therefore stronger than pointer stability: captured computation may consume a routed slot only through a mapping published for a ready lease, while the shared lane retains a fixed owner and allocation throughout replay.

4.4 Semantics-Preserving Router Decomposition

The staging decision is known only after routing, yet a monolithic fused-MoE operation hides expert selection from the graph partitioner. LatchMoE therefore relocates, rather than reimplements, the selection step into a captured router operation. For an external-logit layer, the operation consumes the original router logits. For an internal-gate layer, it first invokes the same gate on the same hidden states. It then calls the native selector with the layer's top-k, grouping, scoring function, renormalization flag, correction bias, routed scaling factor, and logical expert count.

The router emits the original top-k IDs and weights. The eager boundary uses only IDs to plan residency and cannot edit either tensor. Captured expert computation injects both into the native apply path, so selection occurs once. The seam relocates routing without approximating its mathematical decision.

Output semantics remain native: the routed-only contract returns one tensor, whereas the shared-expert contract returns the expected fixed pair. The native runner retains shared gating, routed scaling, output transforms, and final addition; LatchMoE owns only residency and ordering.

4.5 Replay-Boundary Staging and Safe Publication

After the captured router completes, the eager staging boundary deduplicates the routed IDs and asks a placement policy for an ordering. The returned plan must be a permutation of the actual routed set: it may prioritize a hit, predicted reuse, or transfer cost, but it cannot add an unrouted expert or omit a routed expert. Hits retain their current leases. For a miss, the transfer engine selects a reassignable slot, increments its version, installs the new logical owner, copies W_{13} and W_2 on a dedicated H2D stream, and records a readiness event.

Publication follows a strict happens-before order. The runtime first checks that the slot still matches the issued lease. It then makes the consumer stream wait for the readiness event, transitions the slot to ready, and only afterward updates the persistent logical-to-slot mapping. Publishing earlier would allow captured compute to observe either partially transferred weights or a completion event from an old owner. Hits and misses leave the boundary through the same address-stable interface, so the subsequent graph piece does not encode how a routed expert became resident.

The stage is an explicit graph split point: capture fixes the router and MLP pieces while permitting routing-dependent host work between them. Pointer checks cover the slot banks and map; the descriptor fixes the operator schema.

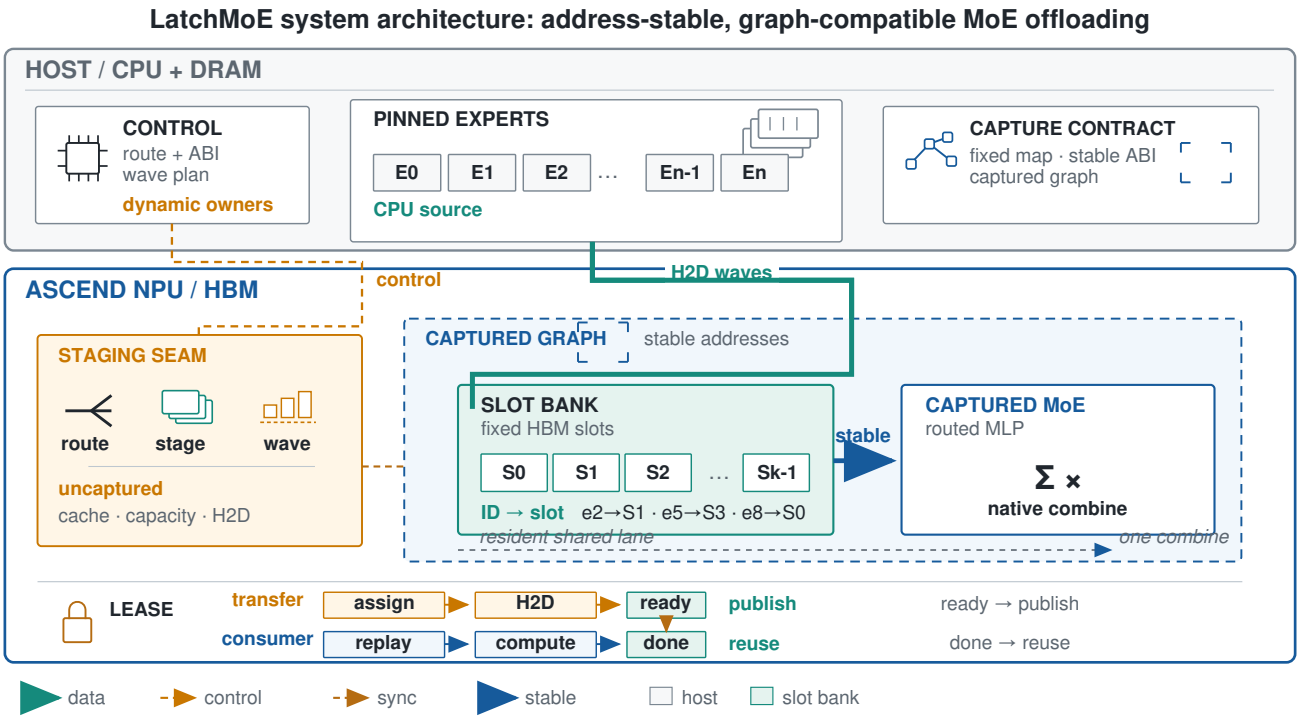

Figure 3. LatchMoE separates dynamic control from graph-visible data. The host control plane computes routing and wave plans while pinned expert weights remain outside the captured region. At the replay boundary, staging checks capacity and transfers misses; the captured NPU region sees only the fixed slot bank, its logical-to-slot map, and stable addresses feeding the fused routed-MoE kernel. The versioned lease at the bottom permits overlap while publishing only after a transfer is ready and reusing a bank only after its consumer lease is done.

4.6 Versioned Transfer–Compute Lifecycle

Address stability alone does not make asynchronous reuse safe. A slot moves through four externally visible states: *empty*, *loading*, *ready*, and *computing*. Only empty or ready slots may be selected for a new owner. Before routed computation begins, the runtime acquires leases for all referenced slots and marks them computing. A compute event protects those leases until the consumer stream finishes; release again verifies owner and version before returning each slot to ready.

Two temporal invariants follow: version validation prevents a stale transfer from publishing, and excluding computing slots prevents in-flight overwrite. The same lease protocol protects transient prefill stage banks: bank b cannot receive wave $k+2$ until the compute event for wave k has released its leases. Lifecycle violations are surfaced as run failures rather than being hidden by fallback.

More precisely, for lease $\ell = (s, e, v)$, replay requires the address of slot s to remain constant. Publishing ℓ requires ready(s, e, v); reassigning s at version $v+1$ requires done(s, e, v). Owner/version validation binds both events to the same ownership interval, so unrelated completions cannot authorize publication or reuse.

4.7 Shared-Aware Capacity-Bounded Composition

Prefill can route a batch of tokens to more distinct routed experts than fit in the slot budget. Let P be the flattened set of routed token–expert pairs, including each pair's original top-k position and weight. LatchMoE partitions P into waves $P_1, \ldots, P_m$ such that each wave requires no more routed experts than a stage bank can hold. Pair membership and routing weights are immutable. Hit-only work may execute from persistent slots, while miss waves use transient banks governed by the same lease rules.

Each wave computes routed outputs while preserving pair positions. After all waves, LatchMoE constructs one global native unpermute index and invokes routed combine once with the original weights and shape. Coverage validation rejects missing or duplicate pairs; single recombination preserves reduction and model-specific postprocessing semantics.

Shared work remains outside the partition and is combined once with the fully reconstructed routed contribution. With multiple transient banks, transfer may populate wave $k+1$ while wave k computes without changing routing or final combination. The storage cost is bounded; Section 6 measures whether compute hides transfer.

4.8 Policy Boundary and Qualification Scope

The data plane exposes only a constrained placement-plan interface: layer ID, complete routed set, and a valid ordering. The default policy preserves first-seen order; another policy may reorder staging but must obey the same capacity, publication, and lifecycle invariants. This boundary lets policy studies reuse one execution substrate without making prediction quality part of the correctness argument.

The design covers single-NPU unquantized BF16 execution, with either no shared expert, an externally represented resident shared lane, or a registered fused shared suffix, and supported external-logit or internal-gate selectors. This ABI alone does not establish correctness or performance for every accepted model; each tuple still requires router/boundary parity, exact outputs, replay evidence, and matched runs. Section 6 reports the qualified boundary.

5 Implementation

We implement LatchMoE as a Python package that registers through vLLM's general plugin entry point. The package integrates with a hook-enabled vLLM-Ascend runtime; the upstream request scheduler and OpenAI-compatible serving API remain unchanged. The host runtime and the Ascend platform plugin are pinned as a compatible stack because the staging boundary changes the internal fused-MoE execution contract rather than the public serving interface.

5.1 Graph integration

The integration decomposes a managed MoE layer into a captured router, the custom `vllm::moe_offload_stage` splitting operation, and a captured expert MLP. The splitting operation returns control to the host after routing so that LatchMoE can stage the active experts. The subsequent graph piece reads the persistent slot weights and logical-to-slot buffer. The runtime locks their data pointers at capture and validates them before replay.

Graph-compatible runs use PIECEWISE ACLGraph. The benchmark validator requires explicit capture and replay records and rejects forced eager execution, eager fallback, graph errors, address changes, stale H2D completion, and slot-version violations. Disabling LatchMoE restores the host runtime's null hooks. The launcher also rejects simultaneous use of the native weight-offload path and LatchMoE because two independent owners of expert storage would violate the slot lifecycle.

5.2 Loading and memory management

CPU-first loading creates managed expert tensors in host memory during model initialization. The host store keeps one contiguous W_{13} tensor and one contiguous W_2 tensor per layer and exposes per-expert views to the transfer engine. It checks shape, dtype, stride, device, layer coverage, and expert coverage before releasing the original device expert banks. Host pinning is used when the runtime supports it and reports failures explicitly.

Automatic configuration derives the resident-layer selection and slot count from the requested offload amount, physical HBM, a KV-cache reserve, and a cap on the fraction of HBM assigned to slot storage. Explicit overrides remain available for controlled sensitivity experiments. The memory ledger accounts separately for the host store, persistent slots, transient prefill stage banks, mapping tensors, full-layer fallback storage, and any original weights that remain allocated.

5.3 Transfers and lifecycle enforcement

The transfer engine maintains a dedicated H2D stream. It validates layout compatibility before copying and batches consecutive host and slot slices when their storage is contiguous. Each asynchronous load returns a readiness object that contains the NPU event and the exact slot leases made consumable by that event. The consumer installs the event dependency before the object marks the leases ready. The runtime then publishes mapping values.

Compute protection uses an event recorded after the expert operation. Slots remain in the computing state until the event completes and every recorded lease still matches its current owner and version. Shutdown drains pending transfer and compute work, closes profiling writers, releases managed storage, and emits a release acknowledgement for the benchmark harness.

5.4 Wave execution and profiling

The prefill path supports a configurable number of stage banks and prefetch depth. The qualified configuration uses two stage banks and depth one. Wave plans carry the original pair offsets needed for one native recombination, and strict validation rejects duplicate or missing coverage. A preflight-qualified overflow decision may select the explicit full-layer mode; NPU, ACL, memory, address, semantic, and lifecycle failures are not swallowed by that path.

Profiling records JSONL events for layer registration, slot ownership, transfers, mapping publication, wave staging and compute, graph replay issue, memory accounting, and service release. Benchmark manifests bind each run to the runtime revisions, model and workload digests, device, command line, environment, graph mode, and raw client/server artifacts. These records support the fail-closed checks used in Section 6.

6 Evaluation

We ask six questions. **Q1** tests semantic fidelity, graph replay, and the lifecycle contract across the three target models. **Q2** establishes baseline feasibility and quantifies the cost of bounded offloading relative to full residency. **Q3** measures capacity-bounded waves against graph-compatible full-layer

staging. **Q4** isolates transfer–compute overlap. **Q5** varies slot capacity. **Q6** accounts for storage, transfer, and unsupported boundaries.

6.1 Experimental Methodology

Platform and software. Experiments use one Ascend 910B2 with 65,536 MiB HBM and tensor parallelism one, hosted by a 192-core Kunpeng-920 server with 2.0 TiB DRAM. Models use BF16 weights. The managed stack pins PyTorch 2.10.0, torch-npu 2.10.0.post2, vLLM commit `ad7125a`, and vLLM-Ascend commit `4806367`. Each run manifest records the complete command, model and dataset digests, source-tree digest, environment, device, graph mode, and service-release custody.

Workloads and controls. The matched baseline, overlap, and capacity campaigns draw from one frozen 64-request ShareGPT prefill-heavy bucket with 1034–1979prompt tokens (median 1394), generate 32 tokens at temperature zero and seed 42, disable prefix caching, and set both client concurrency and `max_num_seqs` to one. The full-layer/multi-wave campaign uses a separate frozen 200-request ShareGPT manifest and 128 output tokens. Every formal unit starts a fresh service. Counterordered comparisons use three start pairs; the paired estimator is the exponential of the mean, across starts, of the median per-request log latency ratio. We report a 10,000-replicate hierarchical bootstrap with seed 20260823: 95% intervals for TTFT effects and 90% intervals for the preregistered TPOT equivalence test (margin ±5%). For the Qwen3 performance campaigns, the 14-GiB Auto-Config manages 12 of the model's 48 MoE layers (one layer in each four-layer group); all four baseline arms and Q3–Q5 use this identical layer selection. Q1 is a separate capability qualification and exercises the model-specific layer counts shown in Table 1.

Fail-closed gates. A performance unit must complete all requests, match complete output-token arrays by request ID, capture and replay PIECEWISE ACLGraph, retain fixed slot and map addresses, publish no stale transfer, violate no owner/version lease, avoid eager fallback and NPU/ACL errors, emit a release acknowledgment, and retain raw client, server, profile, and HBM artifacts. A completed arm that fails exact-token comparison is retained but excluded from latency comparison.

6.2 Q1: Does LatchMoE preserve semantics and replay?

Table 1 covers routed-only Qwen3-30B-A3B, shared-expert GLM-4.7-Flash, and shared-expert Qwen3-Next-80B-A3B-Instruct. Each model is evaluated against the same 11 gate categories: native router parity, native layer-boundary parity, a native oracle where feasible, eager diagnostic parity, exact end-to-end tokens, graph capture and replay, zero eager fallback, overflow composition, decode cache churn, and H2D lease/release ordering. Qwen3-Next's native full-resident end-to-end subcheck is capacity-limited on this device; its native semantics are therefore checked at the router and layer boundary, while its end-to-end comparison uses the eager seam as the reference. The router and layer-boundary checks compare four records per managed layer. Across all three capability tuples, every captured slot/map address validates at replay, every managed layer releases its original expert bank, and the online profiles exercise both overflow waves and H2D leases. Thus, the seam preserves the evaluated native routing and combination contracts rather than merely producing a runnable graph.

This is a capability qualification, not workload breadth: each model uses one short request in each end-to-end mode (four generated tokens) plus a one-token overflow stress request. Qwen3 and GLM compare native, LatchMoE-eager, and LatchMoE-graph outputs. Qwen3-Next cannot materialize its native full-resident path on this 64-GiB device, so its end-to-end gate compares eager-seam and graph outputs; native semantics remain checked at the router and layer boundary.

To expose the graph effect separately from the later multi-wave and overlap comparisons, we also ran a matched one-request qualification diagnostic on Qwen3 with the same 32-slot, 14-GiB configuration. The eager seam and PIECEWISE graph modes produced identical four-token output IDs; the observed TTFT was 2235.66 ms versus 2165.77 ms, and TPOT was 262.04 versus 140.63 ms/token. The diagnostic recorded 48 slot-address validations in each mode and 97 non-synchronizing PIECEWISE replay events. This is direct graph-versus-eager evidence for the qualified path, but it is one request and therefore is not a repeated-start variance estimate, throughput result, or concurrency claim.

6.3 Q2: What baselines are feasible, and what does offloading cost?

We run three counterordered starts of four ACLGraph arms with an explicit 512-MiB KV cache and 32requests per unit: full-resident vLLM-Ascend, its native weight-prefetch path, the legacy layered offload path, and LatchMoE with a 14-GiB target. Full residency and LatchMoE each pass 96/96 exact requests. Native prefetch and legacy layered offload both complete 96/96 requests and capture/replay the graph, but respectively differ from the oracle on 30 and 30 request-token arrays across their three starts. We therefore retain both as unsupported comparison cells and draw no performance conclusion from them.

Full residency is a cost anchor, not a memory-matched offload baseline. Its repeat-median TTFT p50 and TPOT p50 are 179.13 ms and 13.19 ms/token, versus 662.22 ms and 54.50 ms/token for LatchMoE. The paired TTFT effect is +261.28% (95% CI [+240.09%, +277.32%]); the TPOT effect is +309.51% (95% CI [+299.13%, +319.38%]). This is the cost of

Table 1. Three-model semantic and graph qualification. $L/E/S$ denotes managed MoE layers, routed experts per layer, and resident shared experts. Router/boundary counts are exact native comparisons; locks/replays are address locks and observed graph replays; overflow/H2D counts exercise wave composition and transfer leases. Every row passes all 11 gates, including exact end-to-end tokens and zero eager fallback.

Model	$L/E/S$	top-k	Router	Boundary	Locks/replays	Overflow/H2D	Result
Qwen3-30B-A3B	48/128/0	8	192	192	48/97	144/239	Pass
GLM-4.7-Flash	46/64/1	4	184	184	46/94	92/226	Pass
Qwen3-Next-80B-A3B	48/512/1	10	192	192	48/97	144/240	Pass

host-backed expert replacement in the evaluated implementation. In return, the median sampled HBM peak falls from 97% to 80%, while exact output and graph replay remain qualified.

6.4 Q3: Do bounded waves improve full-layer staging?

The graph-compatible full-layer arm materializes the layer working set before compute; the multi-wave arm is identical except that bounded banks execute and recombine the routed work in waves. Three AB/BA/AB pairs each serve 200 requests. All six units complete 1200 requests, match all 153600 output token IDs, capture and replay, and record zero fallback or lifecycle error.

TTFT changes by -35.13% with a 95% CI of [-35.67%, -34.60%]. As a conventional summary, the median across starts of TTFT p50 falls by 35.21%, and TTFT p95 falls by 22.96%. TPOT's paired effect is 0.25% with a 90% CI of [-1.14%, 1.68%], entirely within the preregistered ±5% equivalence margin. The supported result is therefore a prefill-path TTFT reduction without a material TPOT change under this contract.

6.5 Q4: Does overlap contribute causally?

We next hold the wave plan, two stage banks, graph mode, slot capacity, workload, seeds, and source digest fixed, changing only prefetch depth from zero (serial staging) to one (overlap). Three AB/BA/AB pairs serve 64 requests per unit; all 384 requests pass exact-token, graph, lifecycle, and release gates. Overlap changes TTFT by -27.87% (95% CI [-30.18%, -25.24%]); the interval is entirely below zero and passes the preregistered causal-benefit gate. TPOT changes by -4.95% (90% CI [-5.80%, -4.15%]). Its equivalence gate does *not* pass because the lower bound extends beyond −5%; we report the measured decrease rather than calling TPOT unchanged. This one-factor experiment supports overlap's causal contribution, while Q3 measures the complete multi-wave path.

6.6 Q5: How does slot capacity change behavior?

We sweep 16, 32, and 64 physical slots with the same model, 14-GiB target, 512-MiB KV cache, first 32 manifest requests, and runtime source digest. Each point is one fresh start and passes exact-token, graph, capacity, lifecycle, and release gates; latency is therefore descriptive rather than an uncertainty claim. Figure 5 reports the resulting TTFT, average wave count, and persistent-slot storage. From 16 to 64 slots, TTFT p50 falls from 679.36 to 452.67 ms (33.37%), and p95 falls from 754.92 to 529.06 ms (29.92%). Total waves fall from 2808 to 792 and H2D volume from 318.5 to 173.8 GiB, while the persistent slot bank grows from 1.69 to 6.75 GiB and sampled peak HBM rises from 77% to 87%. This point quantifies the intended reuse–storage tradeoff; it is not a cross-workload scaling claim.

Table 2. Audited resources in each model's graph-qualified LatchMoE run. Device storage sums the 32-slot bank, prefill stage banks, and resident shared-expert weights; H2D is the profiled transfer volume for that qualification workload. Original managed expert banks are released in every row.

Model	Host (GiB)	Device (GiB)	H2D (GiB)	Replays
Qwen3-30B-A3B	54.0	14.1	65.5	97
GLM-4.7-Flash	51.8	26.7	22.3	94
Qwen3-Next-80B-A3B	144.0	9.7	152.6	97

6.7 Q6: What resources and boundaries are explicit?

Table 2 accounts for each model's graph-qualified 32-slot run. Host storage scales with the managed expert pool, while device storage depends on expert shape, shared residency, and stage banks rather than total expert count alone. Slot/stage storage is 13.5/0.56 GiB for Qwen3, 25.9/0 GiB for GLM, and 9.0/0.38 GiB for Qwen3-Next; mapping buffers are at most 0.19 MiB. Host-control time is 10.06 s, 0.073 s, and 44.85 s, respectively. These workload-specific values are costs, not a cross-model ranking. Qualification profiles record every original managed bank release; formal performance and Motivation services additionally retain a service-level release acknowledgment. The capability preflight rejects unknown router adapters, fused shared/routed ownership, quantized expert layouts, multi-NPU execution, and shared-expert overlap before capture. These are retained system boundaries: the evaluation establishes a single-NPU BF16 data plane, not silent fallback or support for unqualified tuples.

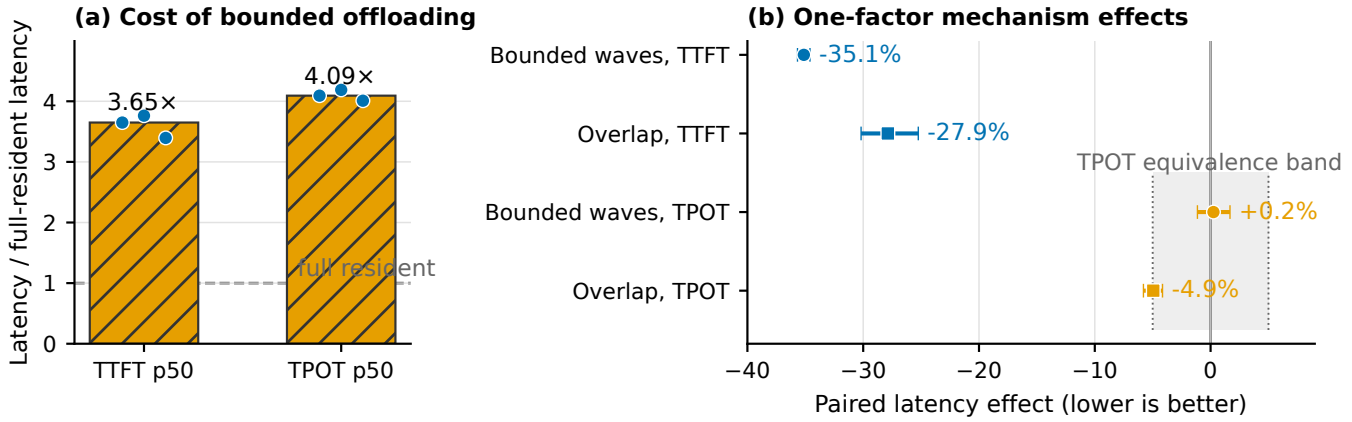

Figure 4. Bounded offloading has an explicit full-resident cost, while its two prefill mechanisms reduce exposed latency. (a) Bars are medians and points are three fresh exact-token, counterordered starts (96 requests per arm); the full-resident reference is 1.0. Native-prefetch and legacy-offload arms are omitted because they fail the exact-token comparability gate. (b) Bounded waves compare multi-wave against full-layer staging (200 requests per unit); overlap compares prefetch depth one against serial depth zero (64 requests per unit). Each contrast uses three counterordered pairs. Points are paired log-ratio estimators; whiskers are 95% CIs for TTFT and 90% CIs for TPOT. Lower is better. All experiments use Qwen3-30B-A3B, one Ascend 910B2, and concurrency one.

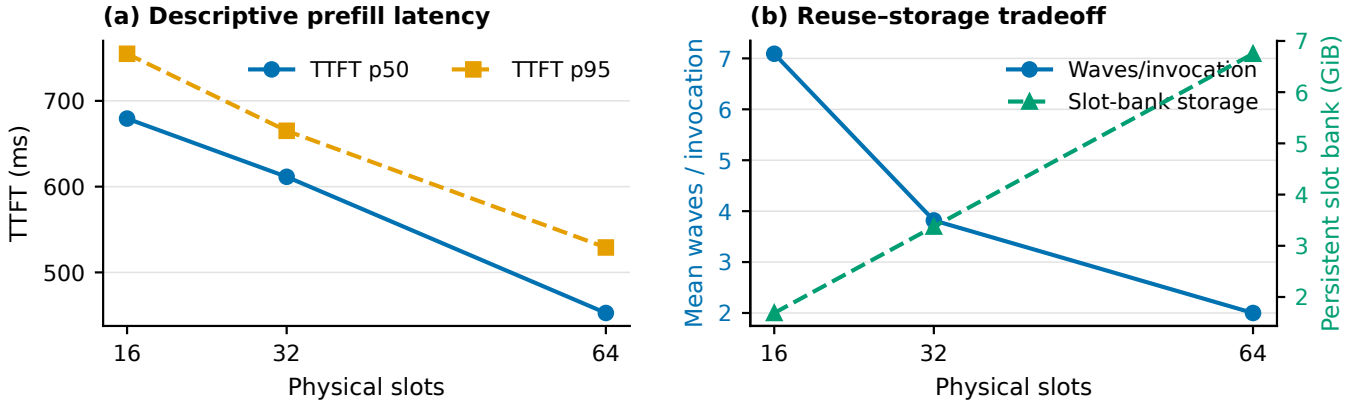

Figure 5. Online slot-capacity sensitivity for Qwen3-30B-A3B with one fresh start per point. All other serving controls are fixed, including a 14-GiB offload target, 512-MiB KV cache, 32 requests, and concurrency one on one Ascend 910B2 with BF16 and tensor parallelism one; prefix caching is disabled. More slots trade persistent HBM for fewer waves; latency values are descriptive.

7 Discussion and Limitations

7.1 Stable storage is the reusable interface

Dynamic identity and graph replay conflict only when identity changes alter graph-visible allocation or execution structure. A fixed storage interface moves dynamism into contents and metadata updated at a controlled boundary. This principle may apply to other bounded working sets with a legal staging boundary and enforceable publication/reuse order; our evidence covers only the implemented MoE path.

Address stability does not prevent stale publication or eviction; owner/version checks and stream dependencies enforce temporal correctness. Static residency sacrifices bounded offload, pointer rebinding requires graph recapture, and eager fallback pays dispatch overhead. LatchMoE fixes slots and stages replacement; its seam and fused operator are hardware-specific.

7.2 Control policy is outside the data plane

LatchMoE accepts an ordered placement plan but constrains it to the actual routed expert set. This boundary lets future caching or prediction policies reuse the fixed slots, transfer engine, wave executor, and graph checks. The current evaluation does not compare placement policies and should not be read as evidence that the default ordering is optimal. Such

a study requires a shared trace, identical storage capacity, and policy-specific hit, transfer, and latency measurements on the same data plane.

7.3 Scope of the current evidence

The semantic and graph qualification covers three unquantized BF16 models on one Ascend 910B2 with tensor parallelism one. It validates native router and layer-boundary parity, exact end-to-end tokens, replay-visible addresses, overflow composition, lease/release records, and zero eager fallback for the three reported capability tuples. Performance experiments are narrower: Qwen3-30B-A3B, a 14-GiB host-offload target, serialized requests, `max_num_seqs=1`, and disabled prefix caching. The matched result shows lower TTFT for multi-wave than full-layer staging under that contract; the one-factor campaign isolates an overlap contribution. The full-resident anchor also exposes the absolute cost of host-backed replacement. Native prefetch and legacy layered offload complete graph-mode requests but fail the exact-token comparability gate, so we retain them without using their latency. Consequently, the reported performance evidence is an internal mechanism comparison against graph-compatible full-layer staging, not a superiority claim over published offloading systems. The slot sweep has one start per point and is descriptive. None of these results establishes a general decode-speedup or multi-request serving claim.

Higher concurrency permits overlapping request leases, prefix-cache reuse adds hits across the staging seam, multi-device execution adds distributed ownership, and quantization changes layouts and operator support. None is covered by the qualified single-request host-to-device protocol.

8 Related Work

Sparse MoE inference. Sparsely gated MoE models increase parameter capacity while activating a subset of experts for each token [3, 5, 15, 16]. Systems for large MoE models also combine routing with model and expert parallelism [11, 15]. LatchMoE does not change the model, router, or expert computation. It addresses the runtime storage interface used when expert weights cannot remain fully resident on one accelerator.

Expert offloading and scheduling. Expert-offloading systems reduce host-to-device movement or its exposed cost through caching, prediction, prefetching, mixed precision, fine-grained placement, and pipelined execution [1, 4, 7, 9, 10, 19, 20, 24]. Recent CPU–GPU hybrid serving also streamloads MoE prefill weights and overlaps CPU and GPU work, while FlexGen coordinates heterogeneous placement for dense models [18, 22]. These systems decide which work or weights to place, retain, and move. LatchMoE addresses an orthogonal execution constraint: changing a cache decision must not change the weight storage bound to replayed expert computation. Its placement interface therefore feeds fixed slots and versioned leases instead of becoming part of the captured ABI.

Graph-based inference execution. Nimble compiles dynamic neural-network execution [17], while FlashInfer provides graph-oriented mechanisms for customizable LLM inference [25]. GraCE uses compiler transformations to increase graph coverage [6]. vTensor decouples computation from physical-page management while exposing a continuous virtual address [23], while ECHO makes dynamic KV-cache eviction and recall graph-friendly [12]. LatchMoE instead manages executable weights selected by a captured router and ordered against transfer and MLP computation. An eager seam exposes a fixed slot/map ABI to the downstream graph.

Positioning. LatchMoE joins dynamic placement with graph replay by separating logical ownership from graph-visible allocation and checking each ownership interval as a lease. UVA alone supplies neither bounded HBM replacement nor ready-before-publication and release-before-reuse ordering.

9 Conclusion

LatchMoE makes routing-driven expert replacement compatible with graph replay by virtualizing dynamic logical owners over address-stable physical slots and ordering each ownership interval with a versioned lease. Its captured-router, eager-staging, captured-MLP seam and capacity-bounded waves pass semantic, graph, and lifecycle qualification across three materially different MoE models. On the scoped single-NPU, concurrency-one Qwen3 workload, bounded waves change paired TTFT by -35.13% in the matched full-layer comparison, and isolated overlap changes it by -27.87%; broader concurrency, quantization, and multi-NPU claims remain outside the qualified boundary. These reductions are a capacity–latency tradeoff rather than a universal speedup: against full residency in Q2, the same workload lowers median sampled HBM peak from 97% to 80% while increasing TTFT and TPOT by 261.28% and 309.51%, respectively.

Acknowledgments

OpenAI Codex was used to inspect and modify code, design and run experiments, analyze results, generate figure and table code, check citations, and assist with manuscript structuring, drafting, and editing. The authors reviewed the resulting code, artifacts, analyses, and text and take responsibility for the work.

References

[1] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. In *Proceedings of the 30th ACM International Conference on*

Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS 2025). Association for Computing Machinery, 715–730. https://doi.org/10.1145/3669940.3707267

[2] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 1280–1297. https://doi.org/10.18653/v1/2024.acl-long.70

[3] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P. Bosma, Zongwei Zhou, Tao Wang, Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. 2022. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 5547–5569. https://proceedings.mlr.press/v162/du22c.html

[4] Zhiyuan Fang, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. 2025. Klotski: Efficient Mixture-of-Expert Inference via Expert-Aware Multi-Batch Pipeline. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 2025)*. Association for Computing Machinery, 574–588. https://doi.org/10.1145/3676641.3716261

[5] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39. https://www.jmlr.org/papers/v23/21-0998.html

[6] Abhishek Ghosh, Ajay Nayak, Ashish Panwar, and Arkaprava Basu. 2026. GraCE: Unlocking CUDA Graphs with Compiler Support for ML Workloads. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, 1927–1947. https://www.usenix.org/conference/osdi26/presentation/ghosh

[7] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1018–1031. https://doi.org/10.1109/ISCA59077.2024.00078

[8] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024). https://doi.org/10.48550/arXiv.2401.04088

[9] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. 2024. SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 6710–6720. https://doi.org/10.18653/v1/2024.acl-long.363

[10] Han Li, Jingwei Sun, Junqing Lin, and Guangzhong Sun. 2026. CommitMoE: Efficient Fallback-Free MoE Inference with Offloading Under GPU Memory Constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 22904–22912. Issue 27. https://doi.org/10.1609/aaai.v40i27.39454

[11] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating Distributed MoE Training and Inference with Lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, 945–959. https://www.usenix.org/conference/atc23/presentation/li-jiamin

[12] Guangda Liu, Wenhao Chen, Chengwei Li, Zhenyu Ning, Jing Lin, Yiwu Yao, Quan Chen, Shixuan Sun, Jieru Zhao, and Minyi Guo. 2026. ECHO: Efficient KV Cache Offloading with Lossless Prefetching for Serving Native Sparse Attention LLMs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, 17–37. https://www.usenix.org/conference/osdi26/presentation/liu-guangda

[13] NVIDIA Corporation. 2026. CUDA C++ Best Practices Guide. NVIDIA CUDA Documentation. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/ Accessed: 2026-08-18.

[14] NVIDIA Corporation. 2026. PyTorch CUDA Graph Integration. CUDA Graph Best Practice for PyTorch. https://docs.nvidia.com/dl-cuda-graph/latest/torch-cuda-graph/torch-integration.html Accessed: 2026-08-18.

[15] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 18332–18346. https://proceedings.mlr.press/v162/rajbhandari22a.html

[16] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=B1ckMDqlg

[17] Haichen Shen, Jared Roesch, Zhi Chen, Wei Chen, Yong Wu, Mu Li, Vin Sharma, Zachary Tatlock, and Yida Wang. 2021. Nimble: Efficiently Compiling Dynamic Neural Networks for Model Inference. In *Proceedings of Machine Learning and Systems*, Vol. 3. 208–222. https://proceedings.mlsys.org/paper_files/paper/2021/file/5b47430e24a5a1f9fe21f0e8eb814131-Paper.pdf

[18] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 31094–31116. https://proceedings.mlr.press/v202/sheng23a.html

[19] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. ProMoE: Fast MoE-Based LLM Serving Using Proactive Caching. *arXiv preprint arXiv:2410.22134* (2024). https://doi.org/10.48550/arXiv.2410.22134

[20] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference. *arXiv preprint arXiv:2411.01433* (2024). https://doi.org/10.48550/arXiv.2411.01433

[21] vLLM Project. 2026. UVA Offloader. vLLM Documentation. https://docs.vllm.ai/en/stable/api/vllm/model_executor/offloader/uva/ Accessed: 2026-08-18.

[22] Wenxin Wang, Yule Hou, Yu Ji, Peng Qu, and Youhui Zhang. 2026. Achieving Cloud-Grade SLOs for Local Mixture-of-Experts Inference through CPU–GPU Hybrid Design. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, 1089–1106. https://www.usenix.org/conference/osdi26/presentation/wang-wenxin

[23] Jiale Xu, Rui Zhang, Cong Guo, Weiming Hu, Zihan Liu, Feiyang Wu, Yu Feng, Shixuan Sun, Changxu Shao, Yuhong Guo, Junping Zhao, Ke Zhang, Minyi Guo, and Jingwen Leng. 2024. vTensor: Flexible

Virtual Tensor Management for Efficient LLM Serving. *arXiv preprint arXiv:2407.15309* (2024). https://doi.org/10.48550/arXiv.2407.15309

[24] Leyang Xue, Yao Fu, Zhan Lu, Chuanhao Sun, Luo Mai, and Mahesh K. Marina. 2024. MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache. *arXiv preprint arXiv:2401.14361* (2024). https://doi.org/10.48550/arXiv.2401.14361

[25] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie W. Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Proceedings of Machine Learning and Systems*, Vol. 7. MLSys. https://proceedings.mlsys.org/paper_files/paper/2025/file/dbf02b21d77409a2db30e56866a8ab3a-Paper-Conference.pdf